Day 1: Introduction to Full Stack Development

Assignment: Concepts, Architecture, and Java Full Stack Technologies

Learning objective

Understand how a web application is built using a frontend, backend, and database, and identify the common technologies used in a Java Full Stack application.

Prepared as a beginner-friendly Day 1 assignment

Task 1 — Full Stack Development

1. What is Full Stack Development?

Full Stack Development means building both the user-facing part of a web application and the server-side part that processes data. A full stack application commonly includes a frontend, a backend, and a database. A full stack developer can work across these layers and understand how they communicate.

2. Frontend vs Backend vs Database

Layer	Purpose	Example technologies
Frontend	The part users see and interact with: pages, forms, buttons, and layouts.	HTML, CSS, JavaScript, React.js
Backend	Receives requests, applies business rules, validates data, and sends responses.	Java, Spring Boot, REST APIs
Database	Stores, updates, and retrieves application data.	MySQL, SQL

3. Client–Server Architecture

In client–server architecture, the client (usually a browser or mobile app) sends a request to a server. The server processes the request and returns a response. The backend may communicate with a database to read or save information. HTTP/HTTPS is commonly used for communication between the frontend and backend, and REST APIs often exchange data in JSON format.

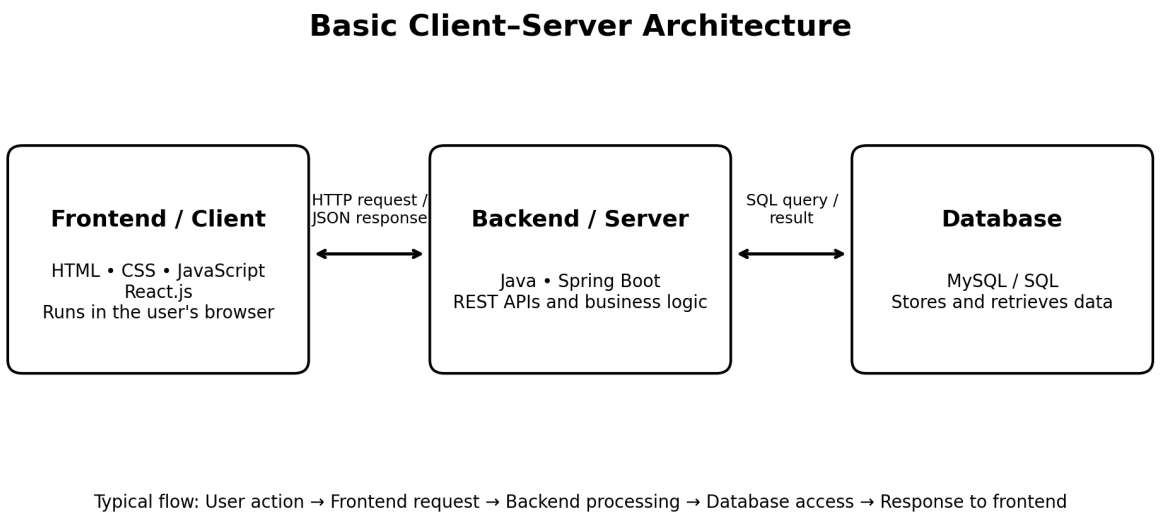


Figure 1. A simplified architecture for a Java Full Stack web application.

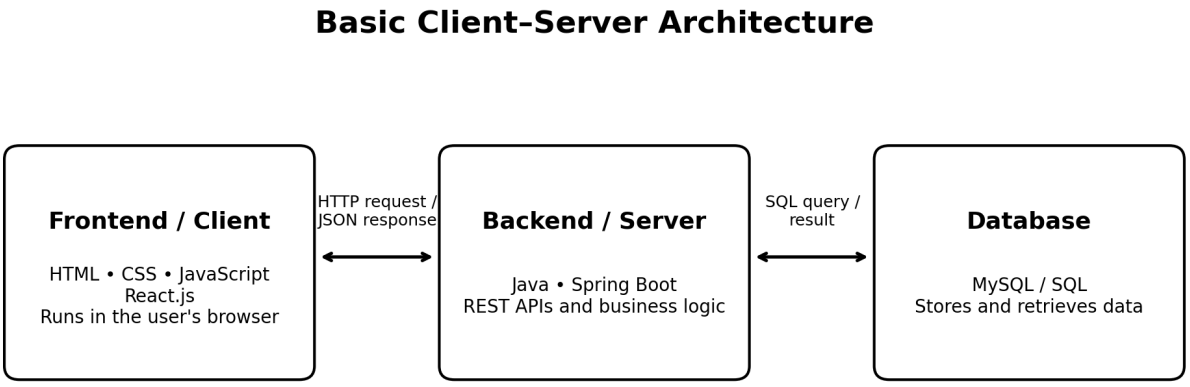
4. How a Web Application Works

1. The user opens the application in a browser.
2. The frontend displays the page and collects the user's action, such as submitting a login form.
3. The frontend sends an HTTP request to a backend REST API.
4. The backend validates the request and runs the required business logic.

5. If data is needed, the backend sends a query to the database.
6. The database returns the result to the backend.
7. The backend sends an HTTP response, often containing JSON, to the frontend.
8. The frontend updates the page and shows the result to the user.

Task 2 — Architecture Diagram

The diagram below shows the three main layers and the communication between them. The frontend sends requests to the backend; the backend reads or writes data in the database and returns a response to the frontend.



Typical flow: User action → Frontend request → Backend processing → Database access → Response to frontend

Communication summary

• Frontend → Backend: HTTP/HTTPS request, usually through a REST API.
• Backend → Database: SQL query or command to read, insert, update, or delete data.
• Backend → Frontend: HTTP response, commonly with a status code and JSON data.

Task 3 — Technologies Used in a Java Full Stack Application

Technology	Role in the application
HTML	Defines the structure and content of a web page.
CSS	Controls the page's design, layout, spacing, and responsiveness.
JavaScript	Adds interactivity and runs frontend logic in the browser.
React.js	A JavaScript library for building reusable, interactive user interfaces.
Java	The programming language used to implement backend logic.
Spring Boot	Helps create Java backend applications and REST APIs with less setup.
REST API	A way for the frontend and backend to communicate using HTTP methods such as GET, POST, PUT, and DELETE.
SQL	A language used to query and manage relational database data.
MySQL	A relational database system that stores application data in tables.
Maven	A build and dependency management tool often used in Java projects.

Example: Java Full Stack technology flow

React.js frontend → calls a **Spring Boot REST API** → backend uses **Java** business logic → accesses a **MySQL database** using SQL → returns a response to React.js.

Conclusion

A web application usually separates the user interface, server-side processing, and data storage into different layers. Understanding how these layers communicate is the foundation of Java Full Stack Development. HTML, CSS, JavaScript, and React.js are common frontend technologies; Java and Spring Boot are used for the backend; and SQL with a database such as MySQL is used to manage data.